Open in app ↗ Sign up Sign In

Search Medium

You have **2** free member-only stories left this month. Sign up for Medium and get an extra one

Ian VonSeggern Follow

Jun 6, 2020 · 13 min read · ✦ · Listen

Save

Note Recognition In Python

As a software engineer in quarantine, I had a lot of time on my hands and was looking for a side project. I wanted to experiment with audio processing in Python, and I just started learning to play the guitar. When reading tabs, I often wonder what the names of the notes are, so I decided to try to determine them programatically. There is plenty of software out there for this task, but I figured it would be fun to see what I could build.

This problem basically boils down to two sub problems. First, detect where the notes are in an audio file, and second, analyze the audio to determine what note was actually played.

I've included a lot of detail about my thought process, but I also know that often the most valuable thing to share with other engineers is your code. So, feel free to check out the Github link at the bottom of the article.

Note Location Detection

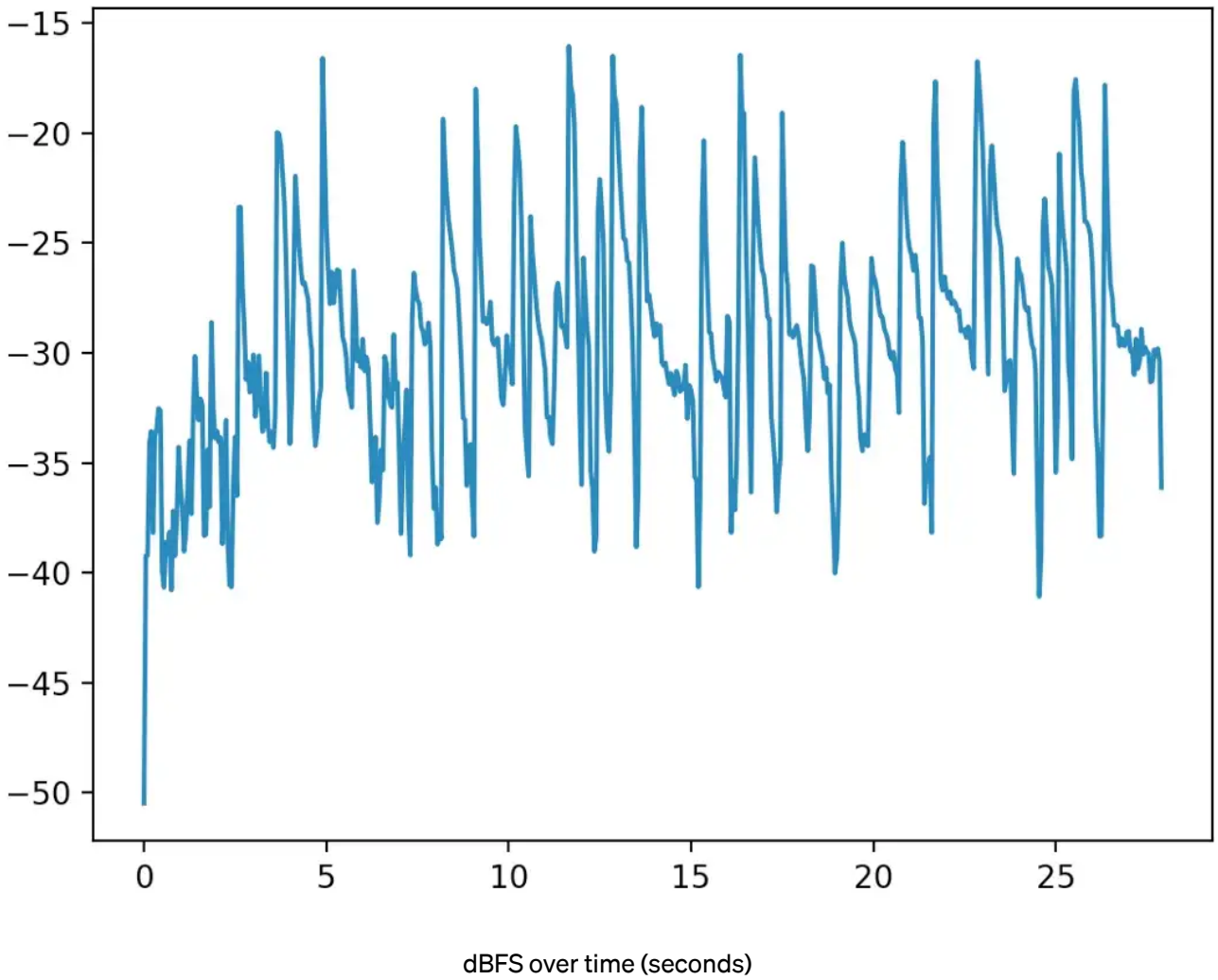
I figured some simple heuristic based on volume might do the trick here.

First, to determine the volume of an audio sample, I found Pydub. It's a sweet library for importing a song and doing basic audio processing. It supports taking slices of audio, increasing the volume, reversing sections, repeating sections, and most importantly, finding the volume of sections. To try this out, I recorded the intro to Bob Marley's "Redemption Song" and wrote the following code.

```
from pydub import AudioSegment

song = AudioSegment.from_file("redemption_song.m4a")

# Size of segments to break song into for volume calculations
SEGMENT_MS = 50

# dBFS is decibels relative to the maximum possible loudness
volume = [segment.dBFS for segment in song[::SEGMENT_MS]]
```

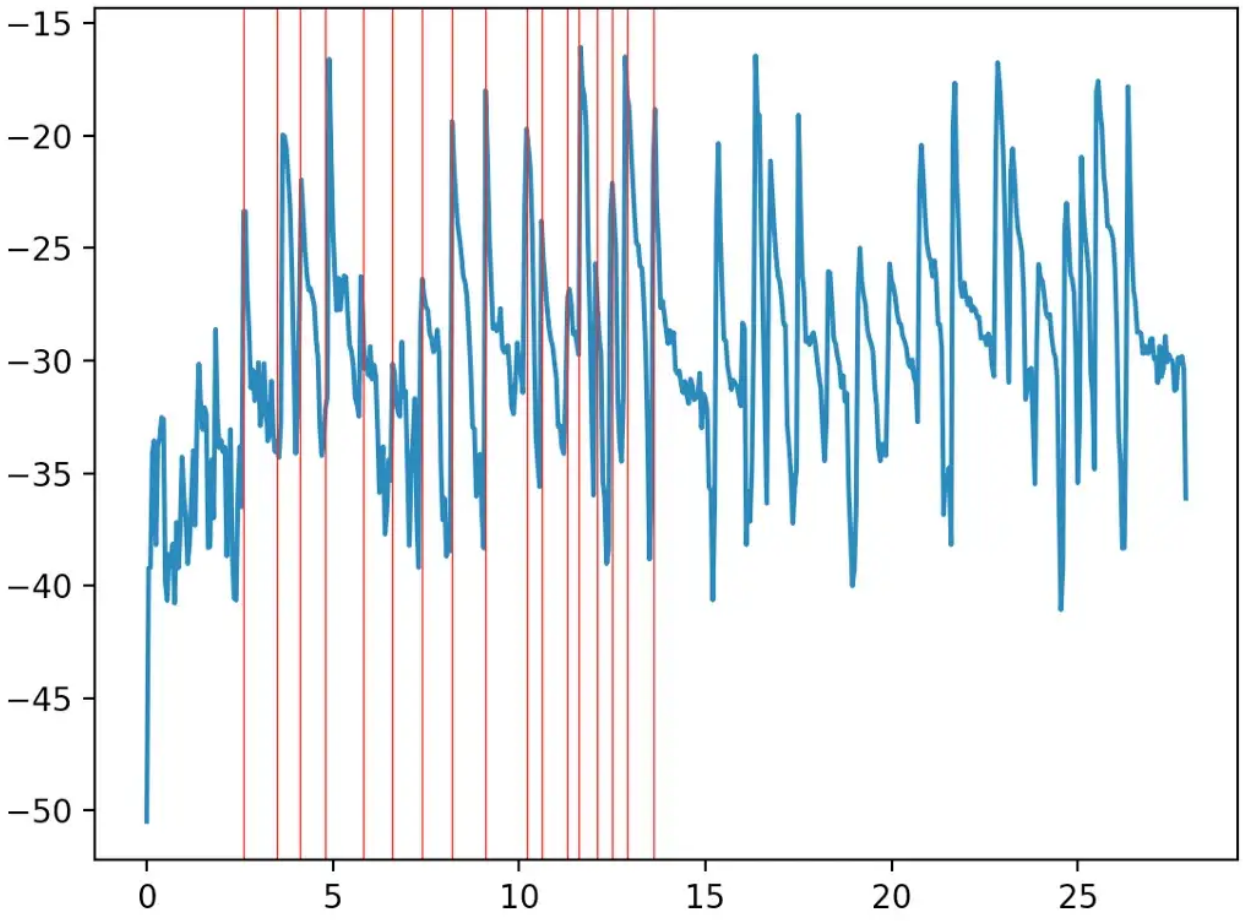
First, I loaded the song into an `AudioSegment`. I broke in into segments using standard python array slicing `song[::SEGMENT_MS]` and called Pydub's `dBFS` function on each segment to get its volume over time. I created a plot with Matplotlib.

```
import matplotlib.pyplot as plt
import numpy as np

x_axis = np.arange(len(volume)) * (SEGMENT_MS / 1000)
plt.plot(x_axis, volume)
plt.show()
```



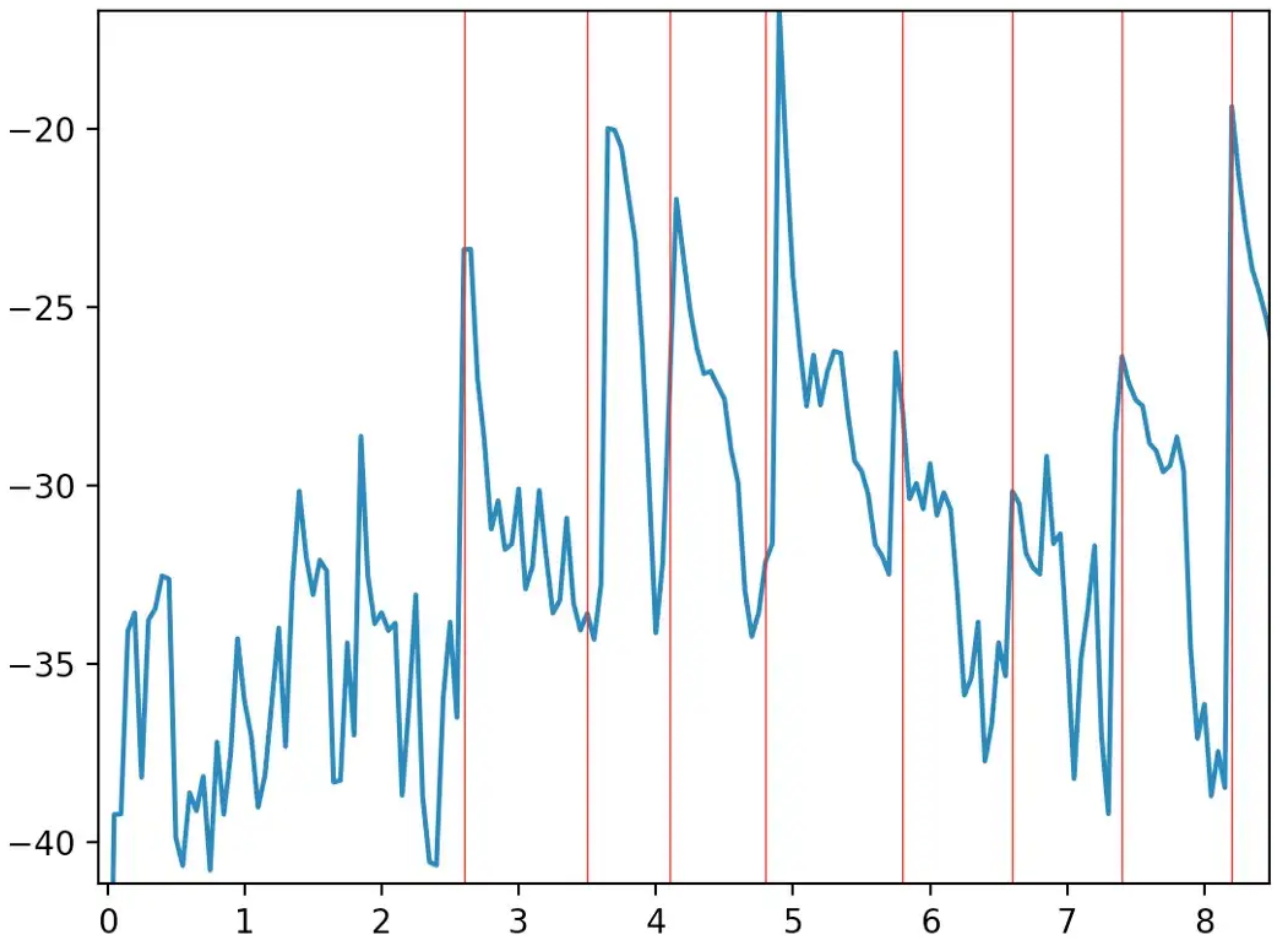
I also went through my recording and manually marked the approximate start times of each note for the first half of the song. I added these to the graph.

```
actual_notes = [2.6, 3.5, 4.1, 4.8, 5.8, 6.6, 7.4, 8.2, 9.1, 10.2,
10.6, 11.3, 11.6, 12.1, 12.5, 12.9, 13.6]

for s in actual_notes:
    plt.axvline(x=s, color='r', linewidth=0.5, linestyle="-")
```



dBFS over time — with note starts

Zooming in a bit:

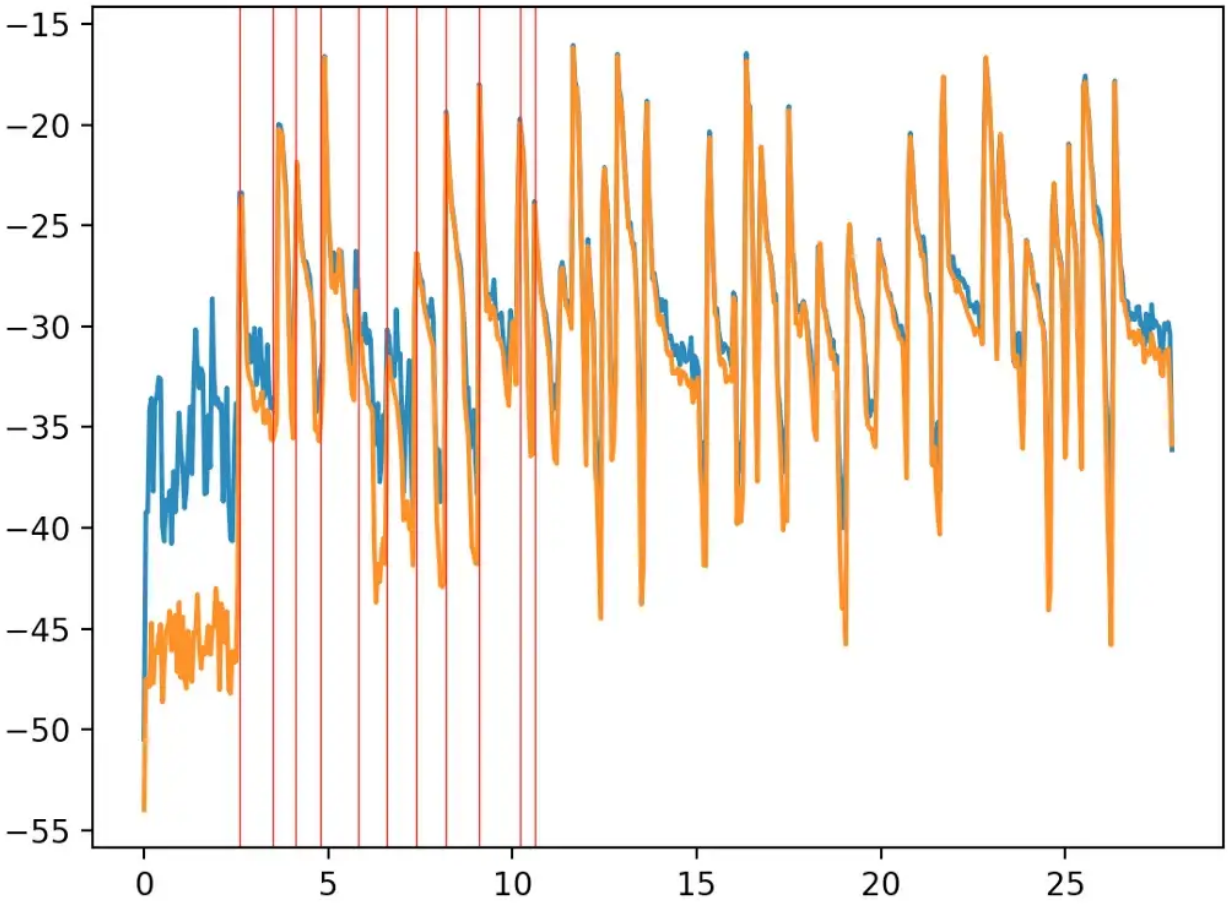


dBFS over time — with note starts

It definitely looks like the start of each note results in a sudden increase in volume. However, it looks like it will be hard to differentiate note starts from noise with a simple heuristic like increase over previous sample and minimum volume. In particular the 6th note was less loud and less dramatic an increase than some of the noise before the first note.

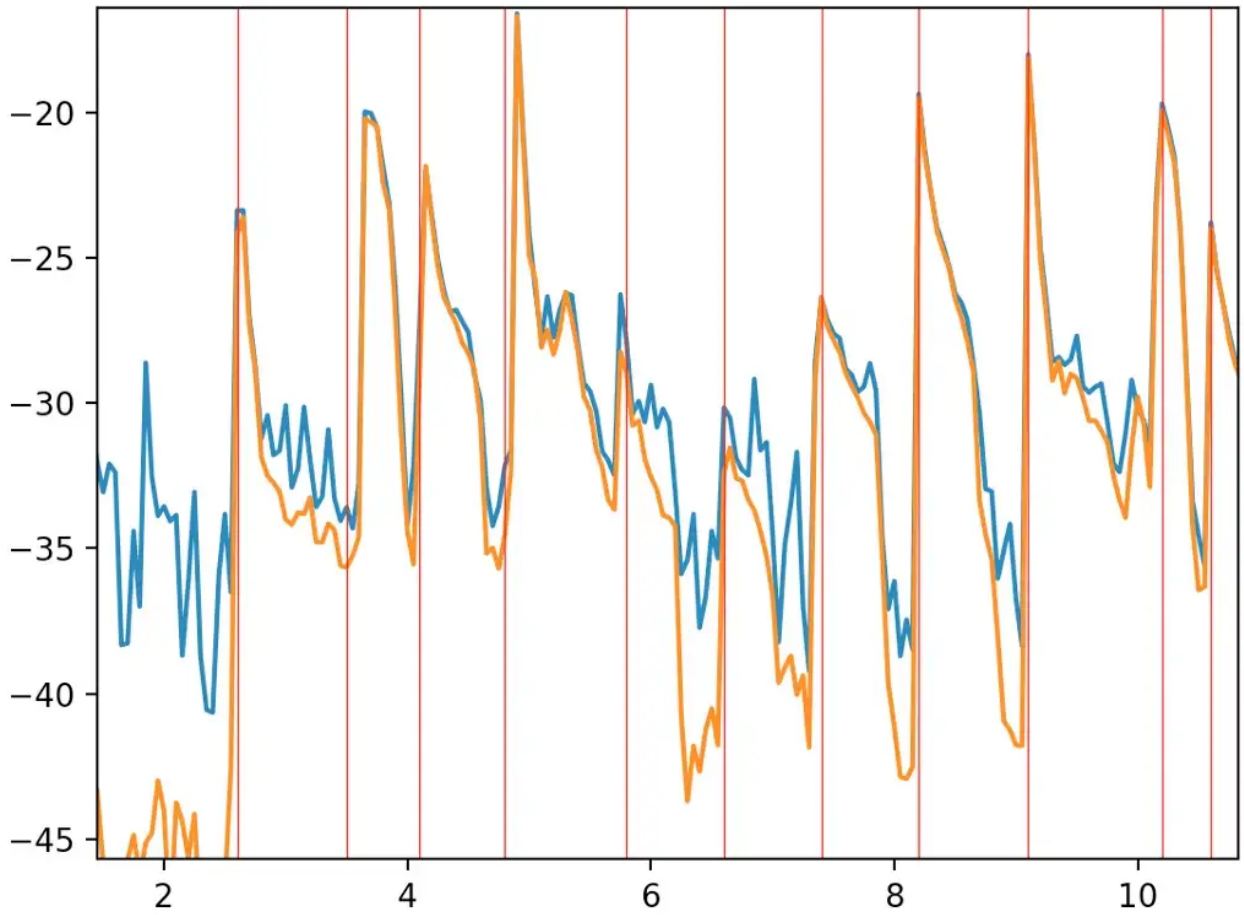
It was pretty quiet when I recorded this, but I wondered if perhaps there was some low or high frequency noise pollution. Thankfully, Pydub has great utilities for low pass and high pass filtering. After a little experimenting, I found that removing much of the frequencies under 80 Hz with a high pass filter helped reduce the noise significantly.

```
# Filter out lower frequencies to reduce noise
song = song.high_pass_filter(80, order=4)
```



dBFS over time — with and without high pass filter

As you can see, the initial noise before the first note is mostly gone now.



dBFS over time — with and without high pass filter

Zooming in, we can see that this filter has also helped remove some of the problematic noise between the notes. Now the 6th note is a lot clearer. The 5th note is probably the most problematic, but even it is a more significant increase from one sample to the next than is found in any of the noise.

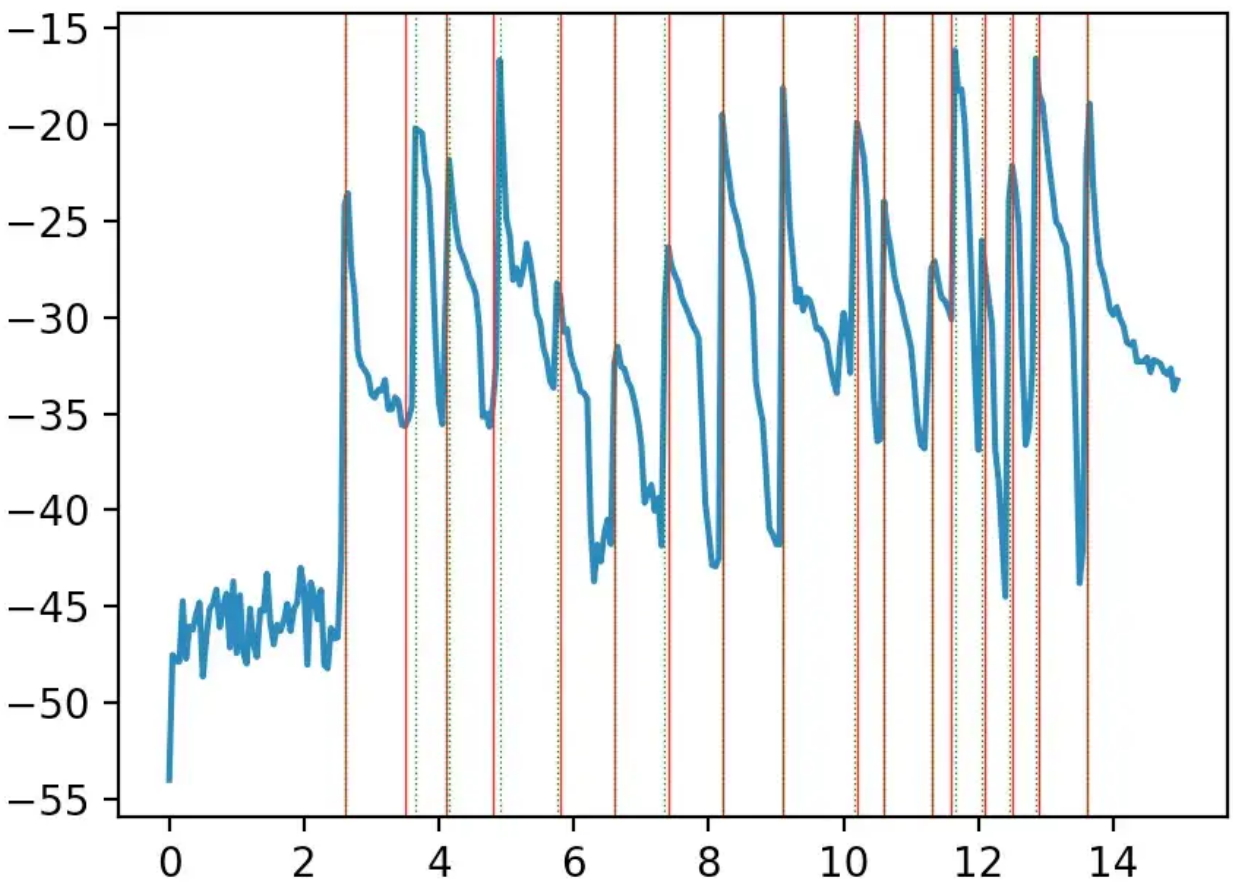
I wrote a quick heuristic that took into consideration the size of the 'cliff' (which I defined as at least an increase of 5 dB from one single sample to the next) and had a minimum threshold for the top of the 'cliff' (-35 dBFS).

```
# Minimum volume necessary to be considered a note
VOLUME_THRESHOLD = -35

# The increase from one sample to the next required
# to be considered a note
EDGE_THRESHOLD = 5

predicted_starts = []
for i in range(1, len(volume)):
```

```
if (
    volume[i] > VOLUME_THRESHOLD and
    volume[i] - volume[i - 1] > EDGE_THRESHOLD
):
    ms = i * SEGMENT_MS
    predicted_starts.append(ms)
```

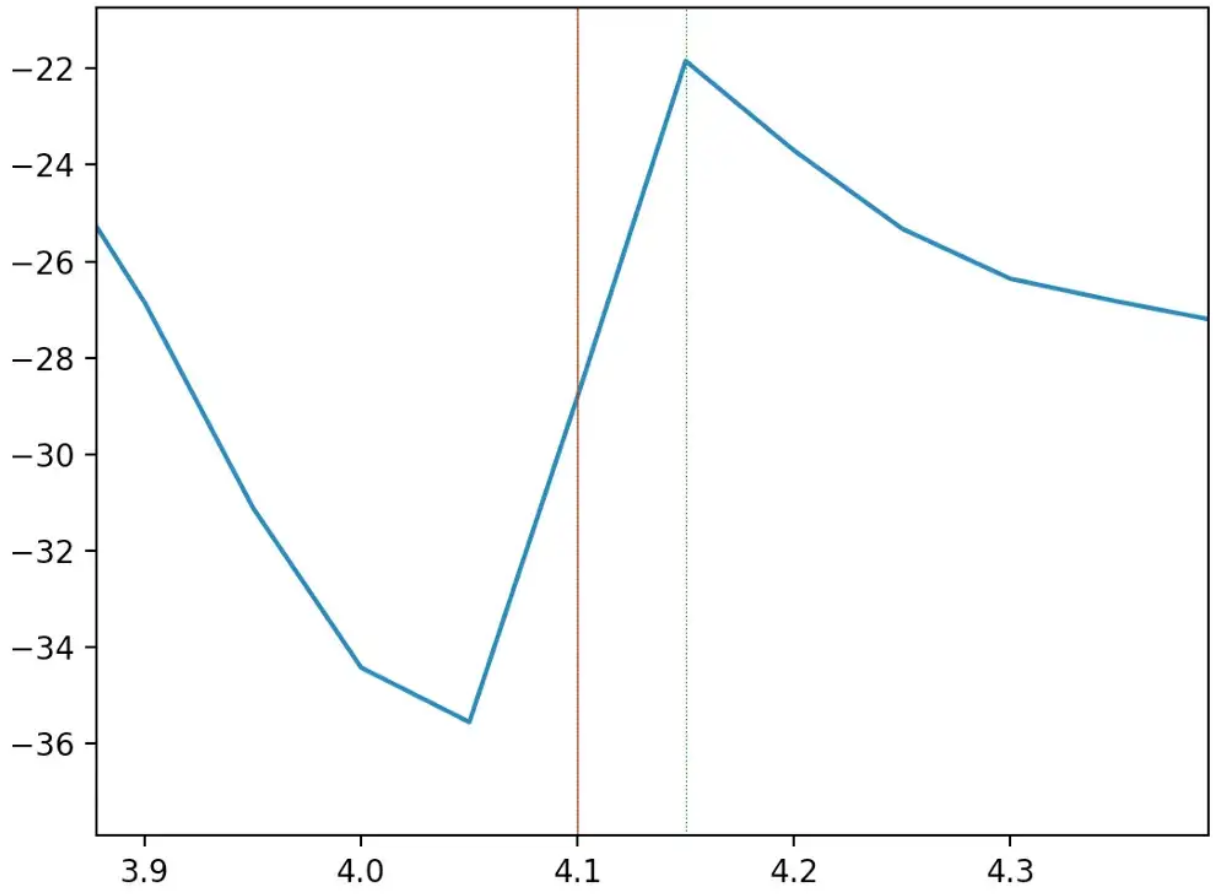
This resulted in the following:

```
Approximate actual note start times (17)

2.60  3.50  4.10  4.80  5.80  6.60  7.40  8.20  9.10 10.20 10.60 11.30
11.60 12.10 12.50 12.90 13.60

Predicted note start times (18)

2.60  3.65  4.10  4.15  4.90  5.75  6.60  7.35  8.20  9.10 10.15 10.60
11.30 11.65 12.05 12.45 12.85 13.60
```



Predicted starts (dotted green) versus actual note starts (red)

These look pretty good! Everything seems to line up except we have one too many notes.



Predicted starts (dotted green) versus actual note starts (red)

The third note's volume increase is split over the 4.05s–4.1s and the 4.1s-4.15s volume segments. I then added a simple rule that notes have to be a minimum distance apart. This resulted in the following code.

```
# Throw out any additional notes found in this window
MIN_MS_BETWEEN = 100

predicted_starts = []
for i in range(1, len(volume)):
    if (
        volume[i] > VOLUME_THRESHOLD and
        volume[i] - volume[i - 1] > EDGE_THRESHOLD
    ):
        ms = i * SEGMENT_MS
        # Ignore any too close together
```

```
        if (
            len(predicted_starts) == 0 or
            ms - predicted_starts[-1] >= MIN_MS_BETWEEN
        ):
            predicted_starts.append(ms)
```

This results in the correct note start times!

```
Approximate actual note start times (17)

2.60  3.50  4.10  4.80  5.80  6.60  7.40  8.20  9.10 10.20 10.60 11.30
11.60 12.10 12.50 12.90 13.60

Predicted note start times (17)

2.60  3.65  4.10  4.90  5.75  6.60  7.35  8.20  9.10 10.15 10.60 11.30
11.65 12.05 12.45 12.85 13.60
```

Obviously this is a pretty basic heuristic and is based on a single sample. If a different sample were a different volume it might not work so well. Also if there was more noise in the background there could be more false positives. Future improvements could include dynamic thresholds, smoothing, and/or comparing multiple samples, but this seems good enough for a first pass.

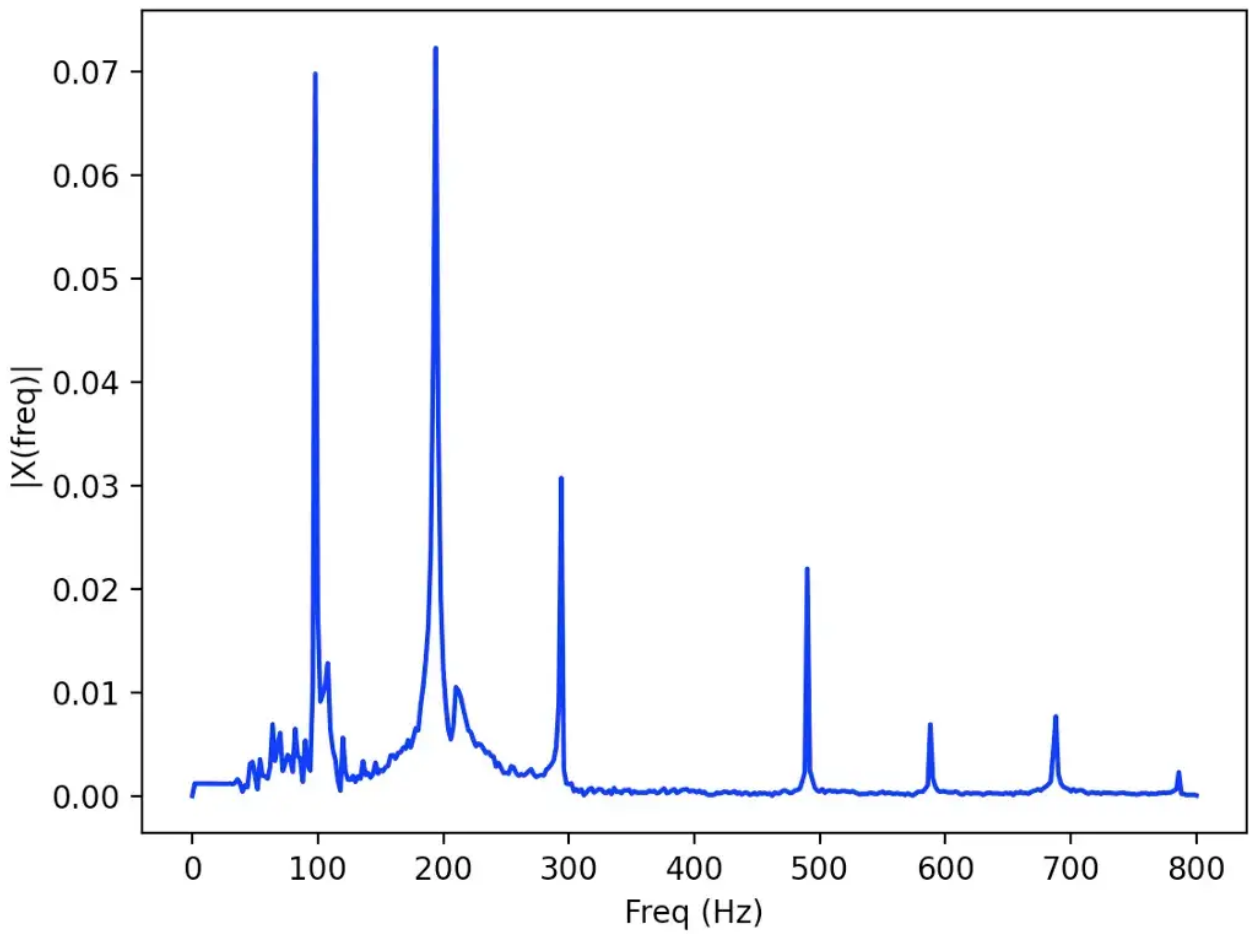
Note Classification

The caveats grow significantly here. Even a quick google search will tell you this is a really hard task and Fourier transforms are insufficient. However, for my rather contrived case, I found I was able to use them to a reasonable level of effectiveness, and I wanted to play with the basics before getting into more complex algorithms.

I had to piece together a couple stack overflow posts to go from a Pydub `AudioSample` to an array of frequency magnitudes. First, I had to convert the `AudioSample` to an array of raw data that I could pass into SciPy's fast Fourier transform (FFT) function. Thankfully, I found an example on StackOverflow from the author of Pydub. The second thing I had trouble with was understanding how to map the index of each value in the output of the FFT array to an actual frequency. Thankfully, there was another great StackOverflow answer which had code to construct another array for each index mapping to the corresponding frequency. That post also did some nice data cleanup, like zero-

normalizing the data, which I went ahead and included as well. I ended up with this.

```
def frequency_spectrum(sample, max_frequency=800):
    """
    Derive frequency spectrum of a pydub.AudioSample
    Returns an array of frequencies and an array of how prevalent that
frequency is in the sample
    """

    # Convert pydub.AudioSample to raw audio data
    # Copied from Jiaaro's answer on https://stackoverflow.com
/questions/32373996/pydub-raw-audio-data
    bit_depth = sample.sample_width * 8
    array_type = get_array_type(bit_depth)
    raw_audio_data = array.array(array_type, sample._data)
    n = len(raw_audio_data)

    # Compute FFT and frequency value for each index in FFT array
    # Inspired by Reveille's answer on https://stackoverflow.com
/questions/53308674/audio-frequencies-in-python
    freq_array = np.arange(n) * (float(sample.frame_rate) / n)  # two
sides frequency range
    freq_array = freq_array[:(n // 2)]  # one side frequency range
    raw_audio_data = raw_audio_data - np.average(raw_audio_data)  #
zero-centering

    freq_magnitude = scipy.fft(raw_audio_data) # fft computing and
normalization
    freq_magnitude = freq_magnitude[:(n // 2)] # one side

    if max_frequency:
        max_index = int(max_frequency * n / sample.frame_rate) + 1
        freq_array = freq_array[:max_index]
        freq_magnitude = freq_magnitude[:max_index]

    freq_magnitude = abs(freq_magnitude)
    freq_magnitude = freq_magnitude / np.sum(freq_magnitude)
    return freq_array, freq_magnitude
```



The magnitude of the frequency response from a 500ms sample of a G note

Next, I took a 500ms sample starting from the first detected note (a G) and graphed it with Matplotlib.

```
plt.plot(freq_array, freq_magnitude, 'b')
```

As you can see there is a peak around 100 Hz, another around 200 Hz, and smaller ones at 300 Hz, 500 Hz, 600 Hz, 700 Hz and 800 Hz. We can use the SciPy `signal.find_peaks` function to find more precise values.

```
peak_indicies, props = scipy.signal.find_peaks(freq_magnitudes,
height=0.015)
for i, peak in enumerate(peak_indicies):
    freq = freq_array[peak]
    magnitude = props["peak_heights"][i]
    print("{}hz with magnitude {:.3f}".format(freq, magnitude))
```

Peaks:

```
98.0hz with magnitude 0.070
194.0hz with magnitude 0.072
294.0hz with magnitude 0.031
490.0hz with magnitude 0.022
```

The good news is these values fit quite well with the physics of a guitar string. The lowest G on a guitar is G2 with a frequency of 98 Hz. Every other value here is close to a multiple of that. Each of these corresponds to an overtone. One issue worth noting here is that while the 1st overtone (196 Hz) is also a G, the next overtone (294 Hz) is actually a D which, in the key of G, is a perfect 5th. Similarly, our last peak (5x the fundamental) is close to 494 Hz. This is a B, the major 3rd.

Based on the above, I wrote a very simple heuristic which predicted the note by determining which note corresponded to the largest peak in the frequency response. I used a helper function that converted a frequency to the nearest note (with a tolerance of 33 cents). This was the following one-liner.

```
get_note_for_freq(freq_array[np.argmax(freq_magnitude)])
```

This is pretty accurate.

```
Actual Notes

['G', 'A', 'B', 'G', 'C', 'E', 'D', 'B', 'G', 'A', 'B', 'D', 'A', 'C',
'B', 'A', 'G']

Predicted Notes
```

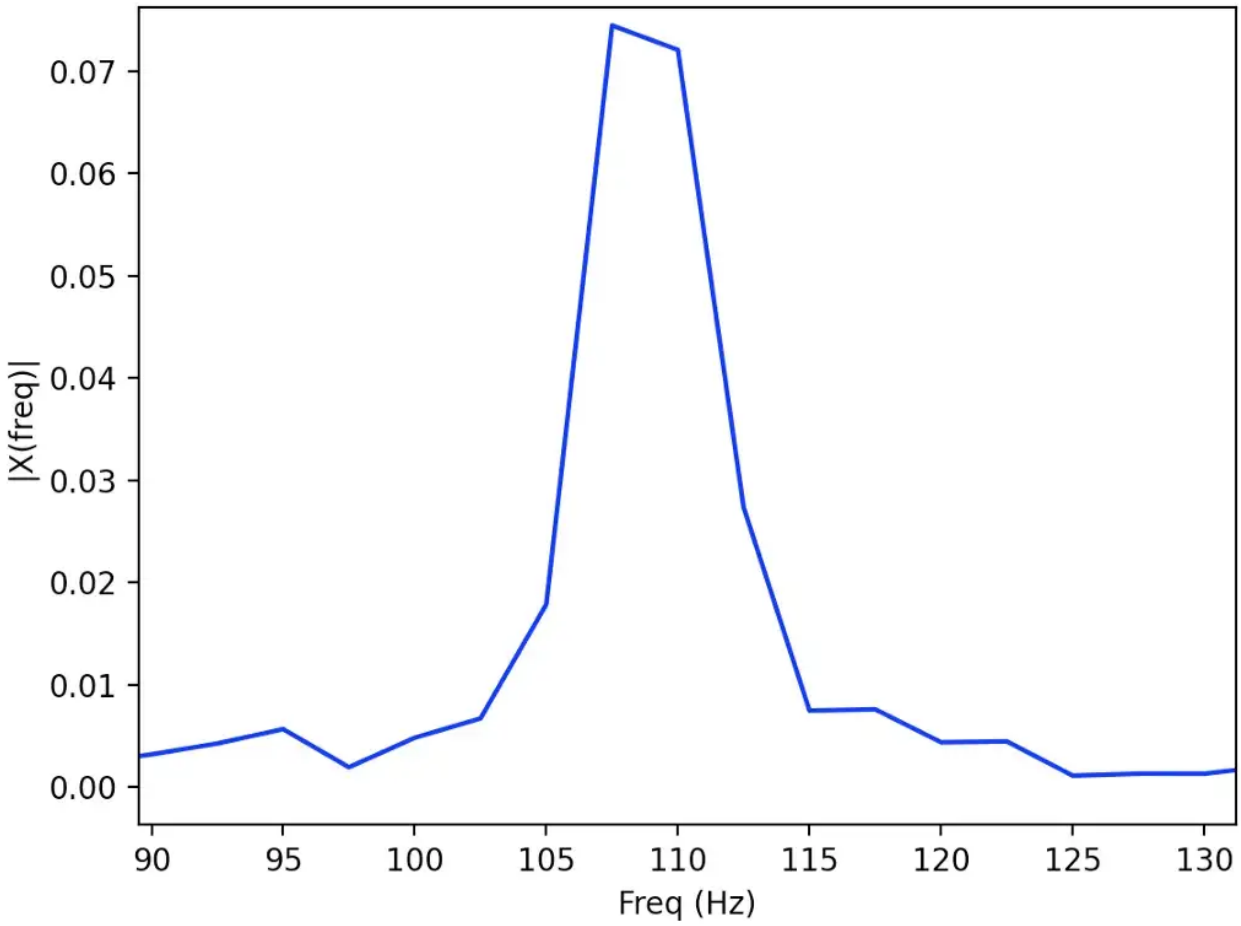
```
['G', 'A', 'B', 'G', 'C', 'E', 'D', 'B', 'G', 'U', 'B', 'D', 'A', 'A',
'B', 'A', 'G']

Levenshtein distance: 2/17
```

This only gets two notes wrong, the 10th note where it predicts 'Unknown' for what should have been an 'A' and the 14th note where it predicts an 'A' which should have been a 'C'.

Let's check out what went on in each case. The 10th note had the following peaks.

```
107.5hz with magnitude 0.074
195.0hz with magnitude 0.022
207.5hz with magnitude 0.016
212.5hz with magnitude 0.017
220.0hz with magnitude 0.064
```

The peak at 107.5 Hz is the highest, but if we graph it, it turns out there is also a large component at 110 Hz, it just happens to be slightly under this peak.



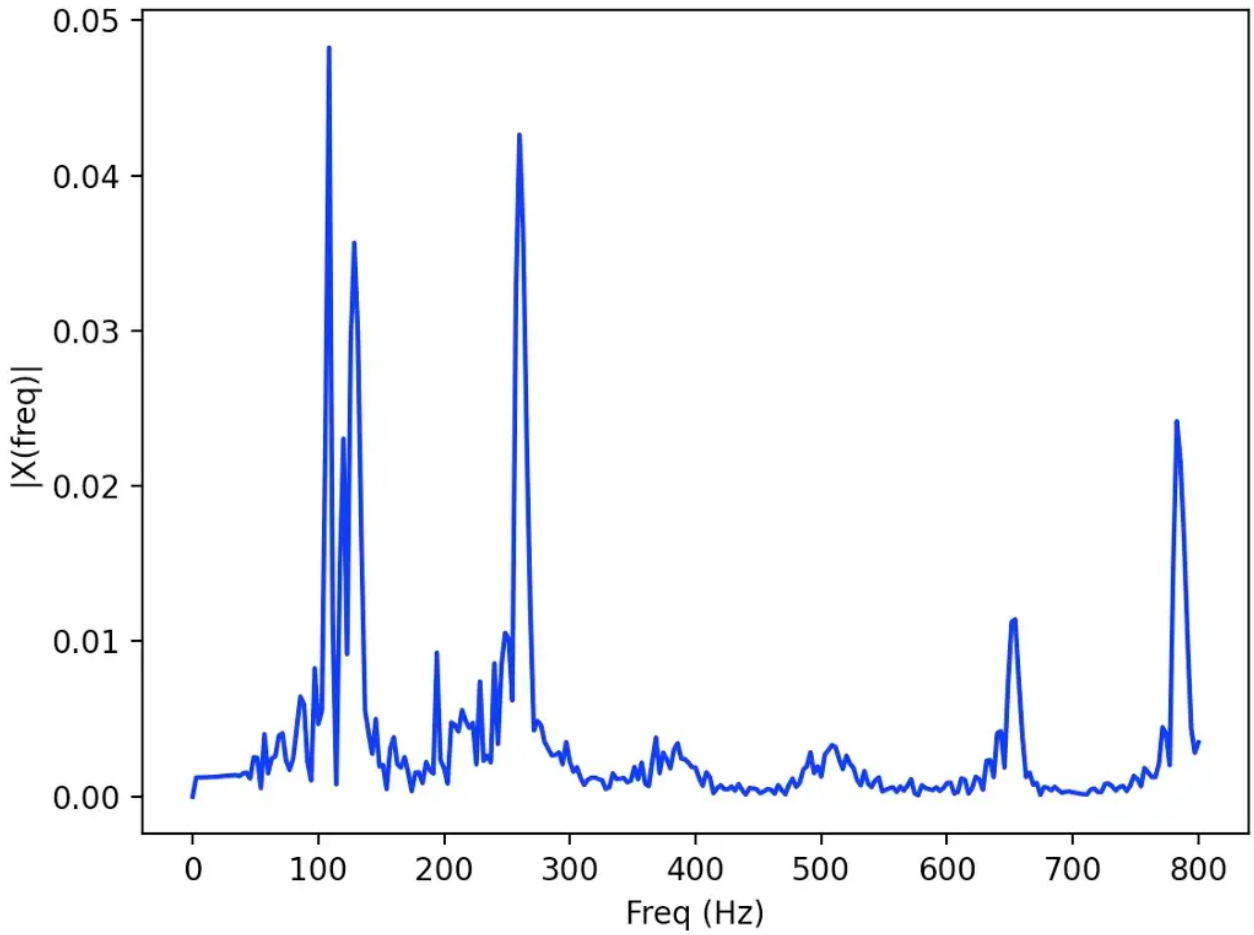
The magnitude of the frequency response for an 'A'

Unfortunately, 107.5 Hz is just outside our tolerance of 33 cents around 110 Hz (this comes out to a lower bound of 107.92 Hz), which is why we claim unknown for this sample.

It's worth pointing out, for the G we looked at above, the frequency response was calculated in intervals of 2 Hz. Here we have 2.5 Hz intervals. This is because the G was slow enough that I was able to capture a full 500 ms sample before the next note started. With this 10th note, I instead only used the sample from the 10.2 second to the 10.6 second because I detected a new note started at the 10.6 second. The shorter the sample the lower the resolution of the FFT. This unfortunately hints at one of the largest weaknesses with this method. Its performance starts to degrade with eighth notes if the song is any faster than 60 bpm and starts to degrade with quarter notes if the song is faster than 120 bpm.

Now let's look at the 14th note, our other failure case.

```
108.6hz with magnitude 0.048
120.0hz with magnitude 0.023
128.6hz with magnitude 0.036
260.0hz with magnitude 0.043
782.9hz with magnitude 0.024
```



The magnitude of the frequency response for a ‘C’ I mistook for an ‘A’

First, the resolution is even worse here. This was a sample of length 350ms resulting in granularity of 2.86 Hz. Looking at this, we do have a strong frequency response near 131 Hz and even more so near 262 Hz. We definitely have some indication there is a C present. However, the tallest peak is near 110 Hz, an A. This is likely due to the prior note which was an A, and it was played only 350 ms earlier. So, not only do faster notes mean shorter resolutions, but they can ‘bleed’ through into the next note. Thankfully, the frequency response for the 220 Hz overtone was pretty muted. So, perhaps accounting more fully for overtones will fix these two missed predictions.

Despite noting multiple potential improvements in the above analysis, I went with a simple improvement for my second attempt. I added up all points with a response of more than 1 percent of the total, attributed those to their notes, and then predicted the note with the highest total attributed frequency response.

```
from collections import Counter

def classify_note_attempt_2(freq_array, freq_magnitude):
    note_counter = Counter()
    for i in range(len(freq_magnitude)):
        if freq_magnitude[i] < 0.01:
            continue
        note = get_note_for_freq(freq_array[i])
        if note:
            note_counter[note] += freq_magnitude[i]
    return note_counter.most_common(1)[0][0]
```

Results:

```
Actual Notes
['G', 'A', 'B', 'G', 'C', 'E', 'D', 'B', 'G', 'A', 'B', 'D', 'A', 'C',
'B', 'A', 'G']

Predicted Notes
['G', 'A', 'B', 'G', 'C', 'E', 'D', 'B', 'G', 'A', 'B', 'D', 'A', 'C',
'B', 'A', 'G']

Levenshtein distance: 0/17
```

Python Pydub Scipy Audio Processing Audio Analysis
Thanks to Sabrina Siu
220 4

Got them all! Sweet. Now let's see how it performs on the second half of the song.

About Help Terms Privacy
Get the Medium app
Download on the App Store GET IT ON Google Play

```
Actual Notes
['G', 'A', 'B', 'G', 'C', 'E', 'D', 'B', 'G', 'A', 'B', 'D', 'A', 'C',
'B', 'A', 'G', 'G', 'A', 'B', 'G', 'C', 'E', 'D', 'B', 'G', 'A', 'B',
'D', 'C', 'B', 'A', 'G']

Predicted Notes
['G', 'A', 'B', 'G', 'C', 'E', 'D', 'B', 'G', 'A', 'B', 'D', 'A', 'C',
'B', 'A', 'G', 'G', 'A', 'B', 'G', 'C', 'E', 'D', 'B', 'G', 'A', 'B',
'D', 'C', 'B', 'A', 'G']
```

```
Levenshtein distance: 0/33
```

Awesome, this algorithm worked on the remaining notes in the song as well. Admittedly tho are basically the same notes played over again (with a single change), but it does confirm th algorithm wasn't overfit exclusively to the sample we were using while developing. It works for audio samples with similar background noise levels, notes in this note range, and notes played at this speed.

Next I tried a new song, "I Walk the Line" by Johnny Cash, which has notes in a similar rang and is played at a similar bpm. However it is a different sequence of notes, and I recorded tl sample at a different time.

```
Actual Notes

['E', 'F#', 'G#', 'A', 'E', 'A', 'A', 'B', 'C#', 'D', 'A', 'D', 'D', 'C#',
'B', 'A', 'E', 'A', 'A', 'G#', 'F#', 'E', 'B', 'B', 'E', 'F#', 'E', 'B',
'D#', 'B', 'A', 'G#', 'F#', 'E', 'B', 'E', 'B', 'E', 'B', 'E', 'B', 'E',
'B', 'E']

Predicted Notes

['B', 'F#', 'G#', 'A', 'A', 'A', 'B', 'C#', 'D', 'A', 'D', 'C#', 'B', 'A',
'E', 'A', 'A', 'G#', 'F#', 'B', 'B', 'B', 'B', 'F#', 'B', 'B', 'D#', 'B',
'A', 'G#', 'F#', 'E', 'B', 'B', 'B', 'B', 'B', 'B', 'B', 'B']

Levenshtein distance: 11/44
```

Not as good. The algorithm missed a few notes simply because they ended up being too qui (the 5th for example). The bigger issue was the number of 'E's it mistook for 'B's. This is likel due to something we noticed earlier. The note B is a perfect 5th above E, and therefore whe you play an E the 2nd overtone has the frequency of a B. I adjusted the algorithm to correct for this by attributing some credit to the note with a 1/3rd the frequency of each observed frequency in addition. I did the same for the other overtones which are registered as differe notes, those at 5 and 7 times the observed frequency.

```
Actual Notes

['E', 'F#', 'G#', 'A', 'E', 'A', 'A', 'B', 'C#', 'D', 'A', 'D', 'D', 'C#',
'B', 'A', 'E', 'A', 'A', 'G#', 'F#', 'E', 'B', 'B', 'E', 'F#', 'E', 'B',
'D#', 'B', 'A', 'G#', 'F#', 'E', 'B', 'E', 'B', 'E', 'B', 'E', 'B', 'E',
```

```
'B', 'E']

Predicted Notes

['E', 'F#', 'G#', 'A', 'A', 'A', 'B', 'C#', 'D', 'A', 'D', 'C#', 'B', 'A',
'E', 'A', 'A', 'G#', 'F#', 'E', 'B', 'B', 'E', 'F#', 'E', 'B', 'D#', 'B',
'A', 'G#', 'F#', 'E', 'B', 'B', 'B', 'B', 'E', 'B', 'E', 'B']

Levenshtein distance: 5/44
```

This improves the results significantly.

Conclusion

This was a really simple first attempt at note recognition, but I was able to get ok results anyway. I'm actually pleasantly surprised how effective a simple FFT was given the pessimis I saw on the internet when I started researching this. Plus, this was also a good chance to learn a bit about the Pydub library and how to take an audio sample and perform a practica FFT on it. You can check out the full code on Github.

I also want to mention, this was also the first time I've taken the time to write up side projec I've worked on. It was more challenging than I expected to document everything and very time consuming. However, I think it was a good process and it is rewarding to have some ki of finished product (even if code is always a work in progress). It also forced me to look mo critically at both my process and the result. I'd like to do more in the future. Thanks for taki the time to read all the way to the end!